

Call Flow

Call hierarchies for implemented major flows. Paths are relative to `BankOne/src/main/java/com/bankone/` unless noted.

Login

```
AuthenticationController.login()
    ↓
AuthenticationService.login()
    ↓
AuthenticationManager.authenticate()
    ↓
CustomUserDetailsService.loadUserByUsername()
    ↓
UserRepository / UserRoleRepository
    ↓
JwtService.generateToken()
    ↓
(LoginAttemptService on failure)
```

Frontend: `AuthService.login()` → store token → navigate `/app/dashboard`.

Get current profile

```
AuthenticationController.me()
    ↓
AuthenticationService.getCurrentUserProfile()
    ↓
SecurityContext → UserRepository / roles
```

Create customer (optional first account)

```
CustomerController.createCustomer()
    ↓
CustomerServiceImpl.createCustomer()
    ↓
CustomerRepository.save()
    ↓
(if accountType present)
AccountService.openAccount() → see Open Account
```

Open account

```
AccountController.openAccount()
    ↓
AccountServiceImpl.openAccount()
    ↓
CustomerRepository.findById()
    ↓
AccountPolicyRepository.findByAccountTypeAndCurrencyCodeAndActiveTrue()
    ↓
validate opening deposit vs policy
    ↓
AccountRepository.getNextOrdinal()
    ↓
AccountNumberGenerator.generate()
    ↓
AccountRepository.save()
```

Frontend (Add Current):

```
CustomerDetail.addCurrentAccount()
    ↓
AccountPolicyService.getActivePolicy()
    ↓
OpeningDepositDialog (if required)
    ↓
AccountService.create / openAccount POST
```

Get active account policy

```
AccountPolicyController.getActivePolicy()
    ↓
AccountPolicyServiceImpl.getActivePolicy()
    ↓
AccountPolicyRepository.findByAccountTypeAndCurrencyCodeAndActiveTrue()
```

Deposit

```
AccountController.deposit()
    ↓
AccountServiceImpl.deposit()
    ↓
AccountRepository.findById()
    ↓
assert ACTIVE + amount > 0
    ↓
increment availableBalance, ledgerBalance, creditCount, timestamps
    ↓
TransactionService.record(..., CREDIT, ...)
    ↓
TransactionRepository.save()
    ↓
AccountRepository.save()
```

Frontend: `AccountList` / deposit dialog → `POST /accounts/{id}/deposit` .

Get account by id

```
AccountController.getAccountById()
    ↓
AccountServiceImpl.getAccountById()
    ↓
AccountRepository.findById()
    ↓
map → AccountResponse
```

Get transactions by account

```
AccountController.getTransactions()
    ↓
PageRequests.of(..., TX_SORT_FIELDS, "createdAt")
    ↓
TransactionServiceImpl.getByAccountId()
    ↓
AccountRepository.existsById() (else IllegalArgumentException)
    ↓
TransactionRepository.findByAccountAccountIdOrderByCreatedAtDesc()
    ↓
map → Page<TransactionResponse>
```

Search accounts

```
AccountController.searchAccounts()
    ↓
AccountServiceImpl.searchAccounts()
    ↓
AccountSpecification.matching()
    ↓
AccountRepository.findAll(spec, pageable)
```

Search customers

```
CustomerController.getCustomers()
    ↓
CustomerServiceImpl.searchCustomers()
    ↓
CustomerSpecification.containsText()
    ↓
CustomerRepository.findAll(spec, pageable)
```

Update account status

```
AccountController.updateAccountStatus()  
    ↓  
AccountServiceImpl.updateAccountStatus()  
    ↓  
AccountRepository.findById() / save()
```

Employee list / create (ADMIN)

```
UserController.*  
    ↓  
UserService.*  
    ↓  
UserRepository / UserRoleRepository / RoleRepository
```

Dashboard summary

```
DashboardController.getDashboard()  
    ↓  
DashboardServiceImpl  
    ↓  
CustomerRepository.count() / AccountRepository / UserRepository  
    ↓  
todayTransactionCount = 0 (stub)
```